tree_clustering.py

User Guide and Algorithm Explanation

Cursor (Composer)
under instruction from Hugh Murrell

August 19, 2026

Abstract

`tree_clustering.py` is a command-line batch tool that implements a k-DPI clustering workflow. For each Newick phylogeny in an input directory it parses days-post-infection (DPI) from the filename, groups tip names, selects a founder tip from the smallest group id, reroots and ladderizes the tree, runs automatic k-DPI bipartition clustering, and writes an annotated linear SVG tree plot. `--no-reroot` skips founder identification and reroot; `--no-ladderize` skips ladderizing. Together they leave the loaded topology unchanged. Optionally it annotates a matching FASTA alignment. After processing a directory it writes a housekeeping CSV and a pie-chart SVG summarising cluster-count distributions across CAP samples. This document is the complete user guide and a detailed description of the algorithms implemented in the script.

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Pipeline . . . 3
1.3 Conventions . . . 3
1.4 Requirements . . . 3

2 Command-line usage **3**
2.1 Synopsis . . . 3
2.2 Positional arguments . . . 3
2.3 Optional arguments . . . 4
2.4 Examples . . . 4

3 Inputs and outputs **5**
3.1 Tree filenames and DPI . . . 5
3.2 Tip name grouping . . . 5
3.3 Per-tree outputs . . . 5
3.4 Batch housekeeping outputs . . . 5

4 Pipeline overview **5**

5 Tree geometry primitives **6**
5.1 Path distance . . . 6
5.2 Cluster depth (LCA height) . . . 6
5.3 Mean pairwise distance and DPI . . . 7
5.4 Expected pairwise tip distance (etd) . . . 7
5.5 Default daily substitution rate . . . 7

6 DPI from filename **7**

7 Grouping **8**

8 Founder selection **8**

9 Reroot on founder **8**

10 Ladderize by depth **8**

11 k-DPI Auto clustering **8**
11.1 Pre-noise (long branches) . . . 8
11.2 Partition tree . . . 9
11.3 Candidate bipartitions . . . 9
11.4 Which leaf to split (Auto) . . . 9
11.5 Auto loop . . . 9

12 Cluster assignment and noise **9**

13 Optional FASTA annotation **11**

14 SVG export **11**
14.1 Layout . . . 11
14.2 Scale modes . . . 11
14.3 Per-cluster bars . . . 11
14.4 Colours and legend . . . 12
14.5 Example plots . . . 12

15 Housekeeping pie chart **14**

16 Default parameters (summary) **15**

17 Failure modes and exit codes **15**

18 Worked numerical sketch **15**

19 File map **15**

20 Versioning note **15**

A Colour palette (clusters) **16**

1 Introduction

1.1 Purpose

This Python script enables the clustering of tips in the phylogenetic tree of an early viral outbreak when the user has an estimate of the number of days post infection (DPI) of the sample that generated the tree. The code makes use of an estimated daily substitution rate (DSR) to cluster tips that do not break the user-supplied DPI threshold. The algorithm continues to split clusters of tips until half the mean pairwise distance of a cluster is less than the DPI yardstick. In so doing the algorithm will automatically generate k clusters and so we name it the k-DPI clustering algorithm.

1.2 Pipeline

The script batch-processes phylogenetic trees with a fixed pipeline:

1. load a phylogenetic tree;
2. group sequences by a name field;
3. reroot on a founder;
4. ladderize;
5. run k-DPI Auto clustering;
6. export a DPI-aware tree SVG (and optionally annotate FASTA).

It is intended for batch analysis of many participant trees (typically CAP cohort Newick files) with consistent parameters.

1.3 Conventions

Notable defaults and design choices:

- default grouping uses delimiter `_` and field 3;
- k-DPI bipartition objective $\min \max(\text{depth}_1, \text{depth}_2)$ with both children required to meet minimum cluster size;
- DPI from mean pairwise tree-path distance;
- cluster / noise naming `_cl-`x / `_cl-n`;
- linear SVG export with AUTO or legacy DPI branch scaling.

Founder selection uses the *tree-path medoid* of the group with the smallest group id.

1.4 Requirements

- Python 3.9+ (uses `from __future__ import annotations`);
- standard library only (no Biopython or other third-party packages).

2 Command-line usage

2.1 Synopsis

```
python3 tree_clustering.py INPUT_DIR OUTPUT_DIR [options]
```

2.2 Positional arguments

`INPUT_DIR` Directory containing Newick tree files. Recognised extensions: `.nwk`, `.newick`, `.tree`, `.tre`, `.treefile`, `.txt`.

`OUTPUT_DIR` Directory for SVG trees, optional annotated FASTA, `housekeeping.csv`, and `housekeeping.svg`. Created if missing.

2.3 Optional arguments

Option	Default	Meaning
`--delimiter`	_	Character splitting tip names into fields for grouping.
`--field`	3	1-based field index used as the group label.
`--dsr`	6.5×10^{-5}	Daily substitution rate (mutations / site / day); see Section 5.5.
`--min-size`	1	Minimum allowed size of each child after a bipartition. Singletons are still reclassified as noise at assignment.
`--max-splits`	63	Maximum number of Auto splits (upper bound on $k-1$).
`--scale`	`auto`	SVG branch scale: `auto` (fit tree + target marks) or `dpi` (legacy: expected depth at 520/16 px).
`--fasta-dir`	(none)	Directory of FASTA files with the same basename as each tree. When set, tip names and FASTA headers are annotated with cluster suffixes.
`--no-long-branch-noise`	off	Disable removal of tips whose incoming branch exceeds etd (expected pairwise tip distance; etd = 2 × dsr × DPI).
`--no-reroot`	off	Skip founder identification and reroot; cluster the loaded rooting (no `[Founder]` highlight).
`--no-ladderize`	off	Skip ladderize by depth; keep the loaded (or post-reroot) tip order.

2.4 Examples

```
# Minimal batch run
python3 tree_clustering.py trees_in/ trees_out/

# Default grouping and annotate FASTA
python3 tree_clustering.py trees_in/ trees_out/ \
  --delimiter _ --field 3 \
  --fasta-dir fasta_in/

# Legacy DPI 1/16 scale, no long-branch pre-noise
python3 tree_clustering.py trees_in/ trees_out/ \
  --scale dpi --no-long-branch-noise

# Skip founder identification and reroot (still ladderize)
python3 tree_clustering.py trees_in/ trees_out/ --no-reroot

# Skip ladderize (still reroot on founder)
python3 tree_clustering.py trees_in/ trees_out/ --no-ladderize

# Cluster the loaded topology unchanged (former --as-is)
python3 tree_clustering.py trees_in/ trees_out/ --no-reroot --no-ladderize
```

3 Inputs and outputs

3.1 Tree filenames and DPI

DPI days are parsed from the tree basename with the regular expression

$$\texttt{dpi[+-]}(+)\text{—}$$

(case-insensitive). If several matches occur, the *largest* integer is used. If none are present, DPI defaults to 14.

Examples:

- `C002_CAP008_dpi+79.nwk` → DPI = 79;
- `x_dpi-14_dpi+100.nwk` → DPI = 100;
- `plain.nwk` → DPI = 14.

CAP identifiers for housekeeping are parsed with `(CAP\d+)` (e.g. `CAP008`).

3.2 Tip name grouping

Tip names are split on `--delimiter`. Field `--field` (1-based) becomes the group label. Existing cluster suffixes of the form `_cl-`x, `_cl-na`, or `_cl-n` are stripped before grouping.

Example (delimiter _, field 3):

`C002_CAP008_2000_env_pb-AGTCGATC` → group `2000`.

3.3 Per-tree outputs

For a tree with stem S, G groups and C non-noise clusters, the script writes

$$S\texttt{_gr-}G\texttt{_cl-}C\texttt{.svg}$$

into `OUTPUT_DIR`.

If `--fasta-dir` is set, a matching FASTA (S`.fasta` / `.fa` / `.fas` / `.fna`) is required. An annotated copy is written to `OUTPUT_DIR` under the same filename, with headers ending in `_cl-`x or `_cl-n` (noise). Tree tip labels in the SVG are annotated the same way when FASTA annotation is enabled.

3.4 Batch housekeeping outputs

After all trees are processed:

`housekeeping.csv`
: Columns `CAPid`, `Groups`, `Clusters`; one row per successfully processed tree; sorted by numeric CAP id.

`housekeeping.svg`
: Pie chart of CAP samples binned by cluster count: 1, 2, 3–4, 5–8, 9–16, … (empty bins omitted); see Section 15 and Figure 5.

4 Pipeline overview

For each tree file the script executes the following stages in order. `--no-reroot` omits founder selection and reroot (Sections 8–9); `--no-ladderize` omits ladderize (Section 10). Used together they leave the loaded rooting and tip order unchanged. Layout, clustering, annotation, and SVG export still run.

1. **Parse** Newick into an in-memory tree; set parent pointers.

2. **Parse DPI** from the filename (Section 6).
3. **Group** tip names (Section 7).
4. **Founder**: take the group with the smallest group id; choose the tree-path medoid (Section 8) (*skipped* if `--no-reroot`).
5. **Reroot** on the founder (Section 9) (*skipped* if `--no-reroot`).
6. **Ladderize** by subtree depth, ascending (Section 10) (*skipped* if `--no-ladderize`).
7. **Layout** assign root-to-node depths and tip row indices.
8. **k-DPI Auto** clustering (Section 11).
9. **Assign** cluster IDs; reclassify singletons as noise; number non-noise clusters $1..k$ by tip order (Section 12).
10. **Optional FASTA / tip annotation** (Section 13).
11. **SVG export** with AUTO or DPI scale and per-cluster bars (solid target, dashed mean pairwise; Section 14).

Finally, write housekeeping CSV and pie chart.

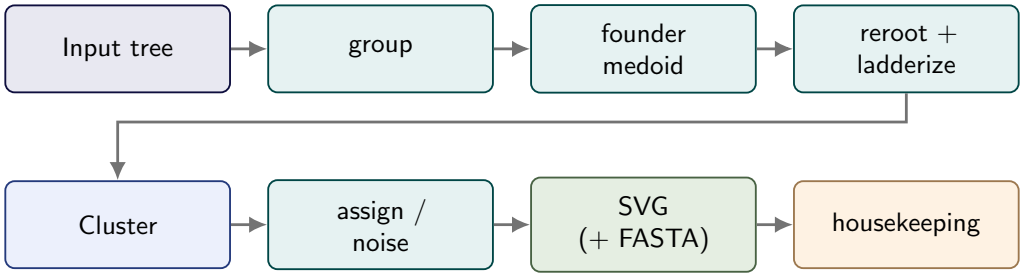


Figure 1: High-level data flow of `tree_clustering.py` (default path). `--no-reroot` skips founder medoid and reroot; `--no-ladderize` skips ladderize.

5 Tree geometry primitives

Let each edge carry a non-negative length $\ell(u)$ stored as the *incoming* length of child u. The root-to-node distance $d(u)$ satisfies

$$d(\text{root}) = 0, \qquad d(u) = d(\text{parent}(u)) + \ell(u).$$

(The implementation includes $\ell(\text{root})$ in $d(\text{root})$ if present; Newick roots usually have $\ell = 0$.)

5.1 Path distance

For tips a, b with lowest common ancestor $\text{lca}(a, b)$,

$$\delta(a, b) = d(a) + d(b) - 2\, d(\text{lca}(a, b)).$$

5.2 Cluster depth (LCA height)

For a tip set T with $|T| \geq 2$ and $L = \text{lca}(T)$,

$$\text{depth}(T) = \max_{t \in T} \big(d(t) - d(L)\big).$$

For $|T| < 2$, depth is defined as 0.

5.3 Mean pairwise distance and DPI

$$\overline{\delta}(T) = \frac{2}{|T|(|T|-1)} \sum_{a<b} \delta(a,b) \qquad (|T| \geq 2).$$

Days-post-infection for a tip set is

$$\widehat{\mathrm{DPI}}(T) = \mathrm{round}\left(\frac{\overline{\delta}(T)}{2\,\mathrm{dsr}}\right). \tag{1}$$

Equivalent form: $\mathrm{dpi} = \overline{\delta}/(2 \times \mathrm{dsr})$.

5.4 Expected pairwise tip distance (etd)

Under a strict-clock star model, the *expected pairwise tip distance* (`etd`) is the expected tree path length between two randomly chosen tips after DPI days:

$$\mathrm{etd} = 2 \times \mathrm{dsr} \times \mathrm{DPI}. \tag{2}$$

(The leading factor of 2 accounts for mutations accruing independently on both lineages from their common ancestor.) Long-branch pre-noise removes tips whose incoming branch exceeds etd. The SVG / scale-bar *target length* used for vertical marks is the one-sided quantity

$$\tau = \mathrm{DPI} \times \mathrm{dsr} \tag{3}$$

(half of etd), i.e. expected root→tip depth for DPI days.

5.5 Default daily substitution rate

The default `dsr` is 6.5×10^{-5} substitutions per site per day. It is obtained from early-infection parameters assembled by Keele *et al.* [1]: reverse-transcriptase error rate $\mu = 2.16 \times 10^{-5}$ per site per replication, generation time $t_{\mathrm{gen}} = 2$ days, and basic reproductive ratio $R_0 = 6$, via

$$\mathrm{dsr} = \frac{\mu\, R_0}{t_{\mathrm{gen}}} = \frac{(2.16 \times 10^{-5}) \times 6}{2} = 6.48 \times 10^{-5} \approx 6.5 \times 10^{-5}. \tag{4}$$

Here μ / t_{gen} is the baseline lineage clock (one replication per generation), and the factor R_0 treats Keele's expanding early-infection regime as raising the *effective* daily substitution rate used when converting tree path length into days ($\widehat{\mathrm{DPI}} = \overline{\delta}/(2 \times \mathrm{dsr})$, $\mathrm{etd} = 2 \times \mathrm{dsr} \times \mathrm{DPI}$). Override with `--dsr` if a different calibration is preferred.

6 DPI from filename

Let M be the set of integers matched by `dpi[+|-]`N in the filename. Then

$$\mathrm{DPI} = \begin{cases} \max M & M \neq \emptyset, \\ 14 & \text{otherwise.} \end{cases}$$

This DPI is used as:

- the Auto split threshold (split only while $\widehat{\mathrm{DPI}} > \mathrm{DPI}$);
- the long-branch threshold via (2);
- the SVG target length τ via (3).

7 Grouping

For each tip name s:

1. strip a trailing cluster suffix matching `delimiter cl-(digits|na|n)`;
2. split on `delimiter`;
3. if field `field` exists and is non-empty, use it as the group label.

Group IDs are contiguous integers $0, 1, \ldots$ ordered by sorted unique labels. Tips lacking the field are excluded from grouping (and therefore cannot be founders).

8 Founder selection

Unless `--no-reroot` is set, the script chooses a founder tip and then reroots on that founder (Section 9). Unless `--no-ladderize` is set, it then ladderizes by depth (Section 10). With both flags, clustering proceeds on the loaded topology.

Let $G^\star$ be the group with the smallest group id (group ids are unique, so this choice is unambiguous). Among tips $T^\star$ of $G^\star$, the founder f is the *tree-path medoid*

$$f = \arg\min_{a \in T^\star} \sum_{b \in T^\star} \delta(a, b). \tag{5}$$

If $|T^\star| = 1$, that tip is the founder.

9 Reroot on founder

Rerooting is omitted when `--no-reroot` is set. Otherwise rerooting places the founder as an outgroup child of a new root:

1. Walk the path from founder leaf to the old root.
2. Detach the founder from its parent; let ℓ_f be the founder edge length.
3. Invert the remaining spine so the former parent becomes the root of the rest of the tree, transferring edge lengths along the flipped path.
4. Create a new root with children {founder, rest} and set both edge lengths to $\ell_f/2$ (midpoint of the founder edge).

Pairwise tip path distances are preserved under this reroot (undirected metric).

10 Ladderize by depth

Ladderizing is omitted when `--no-ladderize` is set. Otherwise, for each node u, define subtree max depth recursively:

$$D(u) = \begin{cases} \ell(u) & u \text{ leaf}, \\ \ell(u) + \max_{c \in \text{children}(u)} D(c) & \text{otherwise}. \end{cases}$$

Children of every internal node are then sorted by ascending $D(c)$ ("shallow subtrees first"). Tip display order (top to bottom) follows this child order after layout.

11 k-DPI Auto clustering

11.1 Pre-noise (long branches)

Unless `--no-long-branch-noise` is set, tips with $\ell(t) >$ etd are removed from clustering and later labelled noise. If all tips would be removed, the filter is abandoned and all tips are kept.

11.2 Partition tree

Clustering maintains a binary partition tree of tip sets ("k-DPI leaves"). Each atomic node stores tip set T, $\widehat{\mathrm{DPI}}(T)$ from (1), and $\mathrm{depth}(T)$.

Initially there is one leaf containing all (non-pre-noise) tips.

11.3 Candidate bipartitions

To split a tip set T with $|T| \geq 2m$ where $m =$ `min-size`:

1. For every phylogeny node, let R be the subset of T descending from it.
2. If $0 < |R| < |T|$, consider the bipartition $(L, R) = (T \setminus R,\, R)$.
3. Accept the candidate only if $|L| \geq m$ and $|R| \geq m$ (no undersized / noise-only children).
4. Score $= \max\big(\mathrm{depth}(L), \mathrm{depth}(R)\big)$.
5. Choose the bipartition with minimal score.

If no candidate exists, a fallback enumerates nonempty proper unions of children of $\mathrm{lca}(T)$ (for polytomies with 2–20 children).

11.4 Which leaf to split (Auto)

Among atomic leaves with size $\geq 2m$, not marked unsplittable, and with $\widehat{\mathrm{DPI}} > \mathrm{DPI}_{\mathrm{target}}$, Auto selects the leaf with largest DPI (ties: larger size).

11.5 Auto loop

Repeat until no eligible leaf remains or `max-splits` is reached:

1. find leaf to split;
2. apply best bipartition (or mark unsplittable and continue);
3. count one successful split.

12 Cluster assignment and noise

After Auto finishes, atomic leaves are classified as follows:

1. Leaves with $|T| < m$ become noise.
2. **Singleton reclassification:** every remaining leaf with $|T| = 1$ is reclassified as noise *before* ID assignment (even when $m = 1$).
3. Remaining non-noise leaves are sorted by the minimum tip-row index of their members (top of the ladderized tree first).
4. They receive contiguous IDs $1, 2, \ldots, k$.
5. Pre-noise tips and all noise leaves map to cluster id -1.

Display / FASTA suffixes:

- noise (id $= -1$): `_cl-n`
- cluster $x \geq 1$: `_cl-`x

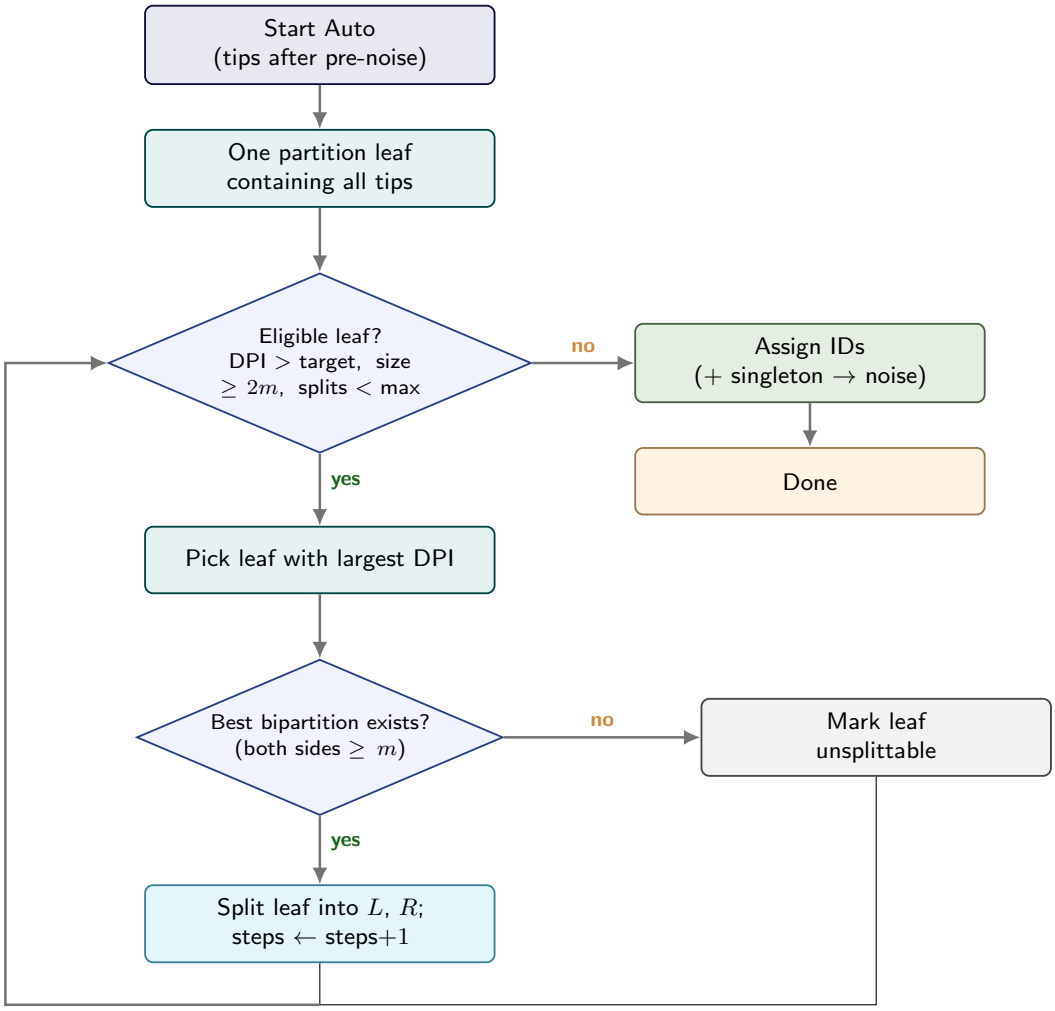


Figure 2: AUTO k-DPI clustering loop. An eligible partition leaf has DPI above the filename target, size at least $2m$ (m = `min-size`), and is not marked unsplittable. Each successful bipartition increments the split counter; the loop also stops when `max-splits` is reached. After the loop, Section 12 assigns cluster IDs and reclassifies singletons as noise.

13 Optional FASTA annotation

When `--fasta-dir DIR` is supplied:

1. Locate `DIR/$S$.{fasta,fa,fas,fna}` for tree stem S.
2. For each FASTA header, strip any old cluster suffix, look up the cluster id, and append `_cl-`x or `_cl-n`.
3. Write the annotated FASTA to `OUTPUT_DIR` (same basename).
4. Rename tree tip labels correspondingly so the SVG shows annotated names.

If no matching FASTA exists, that tree fails with an error (other trees continue).

14 SVG export

14.1 Layout

After clustering, tips are drawn top-to-bottom in ladderized order. An extra top spacer row holds the scale bar above the first tip. Horizontal position of a node is

$$x = 5 + d(u) \cdot x_{\text{scale}}.$$

Branch region width is $W = 520$ px.

14.2 Scale modes

AUTO (default). Fit depth

$$D_{\text{fit}} = \max\big(D_{\text{tree}},\, \tau,\, \max_c\big(d(\text{lca}_c) + \tau\big),\, \max_c\big(d(\text{lca}_c) + \overline{\delta}_c/2\big)\big)$$

where the last two maxima run over non-noise clusters c (so every cluster target bar and mean-pairwise bar remains inside the plot). Then $x_{\text{scale}} = W/D_{\text{fit}}$. When DPI was detected in the filename, the scale-bar label is `(`N `DPI @ dsr)`.

DPI (legacy, `--scale dpi`). Place τ at $W/16 = 32.5$ px: $x_{\text{scale}} = (W/16)/\tau$.

14.3 Per-cluster bars

For each non-noise cluster c with tip set T_c and $L_c = \text{lca}(T_c)$, two vertical lines are drawn in the cluster palette colour of c, spanning only the vertical extent of T_c's tip rows (clamped into the branch region if needed). No text annotation is attached.

Solid target bar. At expected one-sided depth $\tau = \text{DPI} \times \text{dsr}$:

$$x_c^{\text{target}} = 5 + \big(d(L_c) + \tau\big)\, x_{\text{scale}}.$$

This is the Auto stopping threshold: a cluster with $\widehat{\text{DPI}} > \text{DPI}$ still needs splitting.

Dashed mean-pairwise bar. Let $\overline{\delta}_c$ be the mean pairwise tree-path distance among tips in T_c (undefined for a singleton, so no dashed bar is drawn). Place a dashed line (stroke-dasharray 6 4) at the equivalent one-sided depth $\overline{\delta}_c/2$:

$$x_c^{\text{mpd}} = 5 + \big(d(L_c) + \overline{\delta}_c/2\big)\, x_{\text{scale}}.$$

Because $\widehat{\text{DPI}}_c = \text{round}(\overline{\delta}_c/(2 \times \text{dsr}))$, the dashed bar sits at or to the *left* of the solid bar precisely when the cluster does not need a further split ($\widehat{\text{DPI}}_c \le \text{DPI}$). A dashed bar to the *right* of the solid bar flags a cluster that still exceeds the target.

Tip diamonds use half-diagonal 5 px; both cluster bars use stroke width 2.

14.4 Colours and legend

- Tip diamonds: cluster colour (noise grey # 9ca3af).
- Tip labels: group colour (founder highlighted magenta); optional `[Founder]` suffix.
- Legend: groups with $(n, \widehat{\text{DPI}})$ and clusters with $(n, \widehat{\text{DPI}})$, plus noise count.

14.5 Example plots

Figures 3 and 4 show real batch outputs produced by `tree_clustering.py` (AUTO scale). Both trees were rerooted on a founder, ladderized by depth, clustered with k-DPI Auto, and drawn with per-cluster bars (solid target, dashed mean pairwise) and a group/cluster legend.

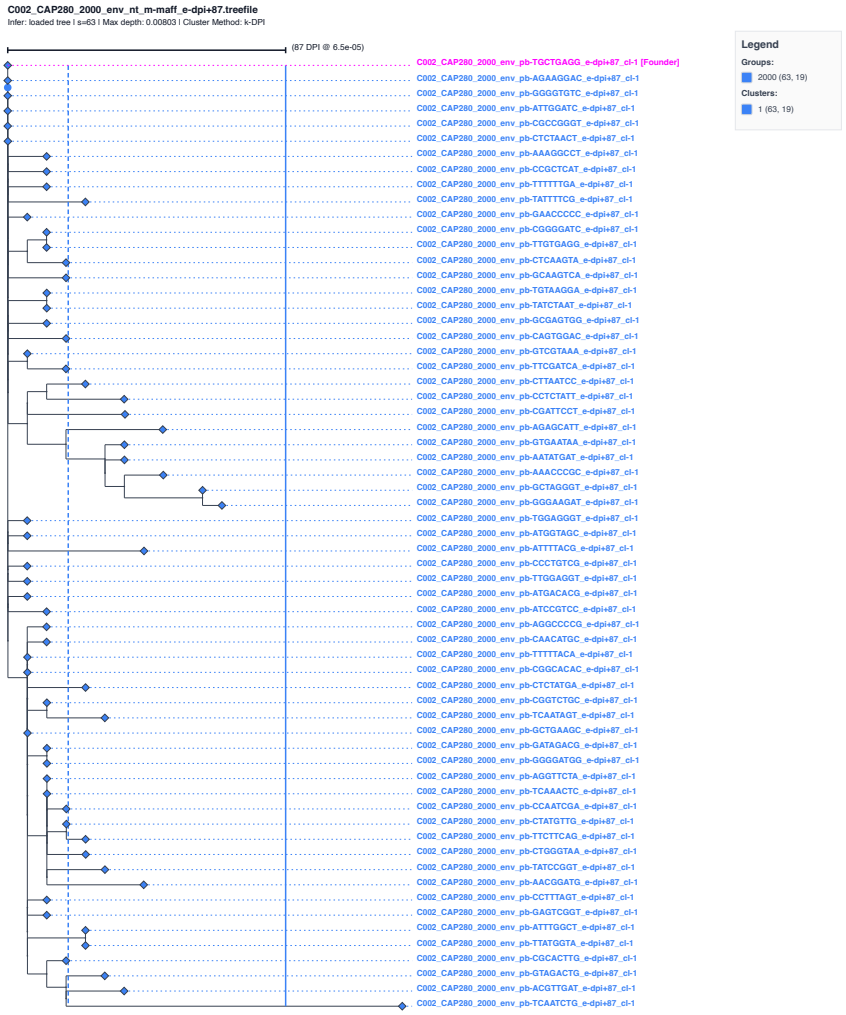


Figure 3: Single-cluster example (`C002_CAP280_2000_...dpi+87_gr-1_cl-1.svg`; s=63 tips, max depth ≈ 0.00803). Filename DPI is 87, with scale bar `(87 DPI @ 6.5e-05)`. All tips form one non-noise cluster, so diamonds share cluster colour 1. A solid vertical target bar in that colour is drawn at $\text{lca} + \tau$ (here the founder-rerooted root), spanning only the cluster tip rows; a dashed bar at $\text{lca} + \bar{\delta}/2$ shows the observed mean pairwise depth. When Auto has finished, the dashed bar should sit at or left of the solid bar. Tip labels use the group colour palette; the founder tip is marked `[Founder]` in magenta and sits as the outgroup after rerooting. The top spacer row holds the scale bar; the right-hand legend lists the single group as `2000 (`n`, DPI)` and cluster `1` similarly. Noise tips (if any) appear grey and are listed under Noise.

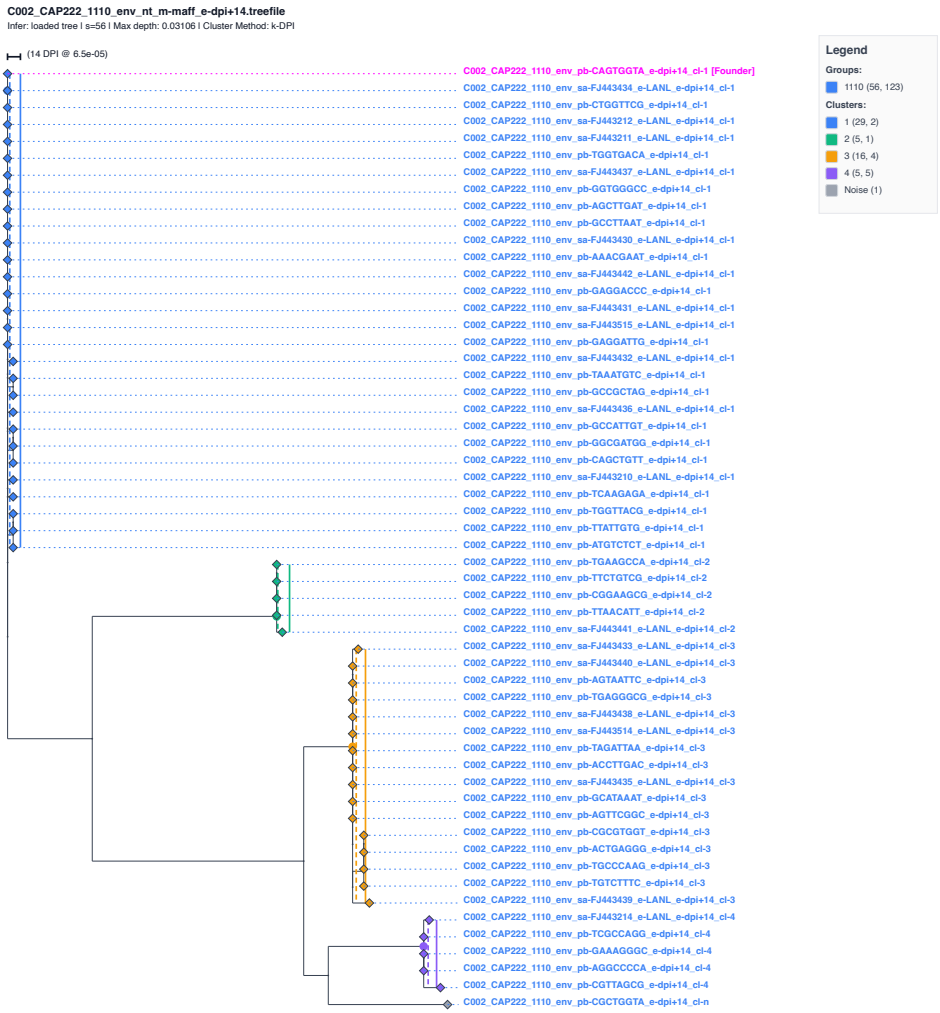


Figure 4: Multi-cluster example (`C002_CAP222_1110_...dpi+14_gr-1_cl-4.svg`; s=56 tips, max depth ≈ 0.03106). Filename DPI is 14, with scale bar `(14 DPI @ 6.5e-05)`. One time-point group is present (`gr-1`, label `1110`). k-DPI Auto produced four non-noise clusters (`cl-4`) plus one noise tip: tip diamonds and cluster bars use palette colours 1–4; noise diamonds are grey. Each cluster has a solid target bar at lca $+ \tau$ and a dashed mean-pairwise bar at lca $+ \overline{\delta}/2$, covering only the vertical span of its tips—so later (deeper) clusters have marks further to the right. Finished clusters have the dashed bar at or left of the solid bar. Tip name colours track the group; the founder is highlighted in magenta with `[Founder]`. Tip suffixes `_cl-`x / `_cl-n` match optional FASTA annotation.

15 Housekeeping pie chart

Each CAP sample contributes its non-noise cluster count C to a bin:

C	Bin label
≤ 1	`1`
2	`2`
3–4	`3-4`
5–8	`5-8`
9–16	`9-16`
⋮	powers-of-two intervals

Only non-empty bins appear as pie slices, ordered by lower endpoint. The chart is written to `OUTPUT_DIR/housekeeping.svg` after the batch run (alongside `housekeeping.csv`).

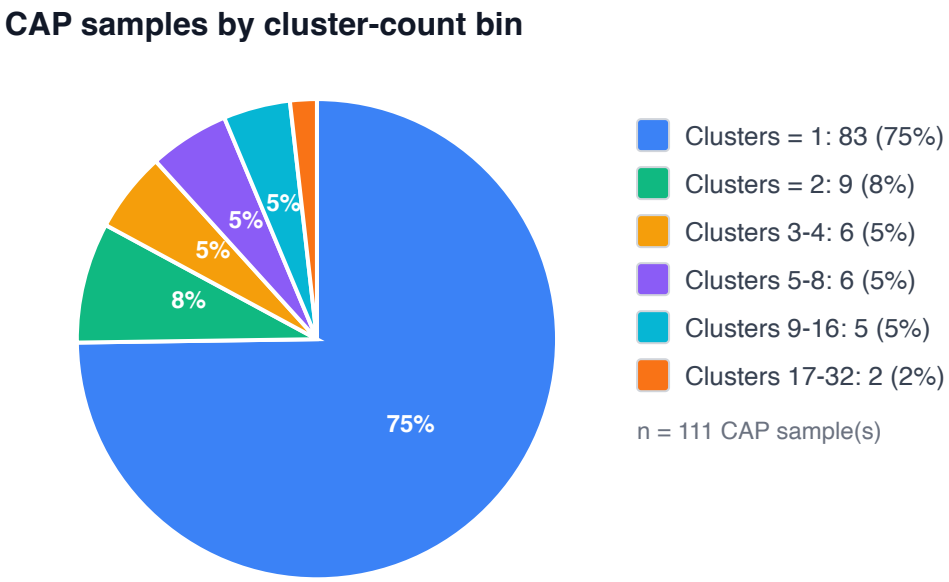


Figure 5: Example housekeeping pie chart (`housekeeping.svg`) for a batch of n=111 CAP samples. Each sample is placed in a power-of-two cluster-count bin (1, 2, 3–4, 5–8, ...); empty bins are omitted. Here most samples remain a single cluster (75%), with smaller slices for 2, 3–4, 5–8, 9–16, and 17–32 clusters. Slice colours follow the same palette as the tree SVGs; the legend lists count and percentage per bin.

16 Default parameters (summary)

Parameter	Default
Daily substitution rate (`dsr`)	6.5×10^{-5}
DPI if absent from filename	14
Group delimiter / field	_ / 3
Min cluster size	1
Max Auto splits	63
SVG scale mode	`auto`
Branch plot width	520 px
Long-branch pre-noise	enabled

17 Failure modes and exit codes

- Per-tree failures (parse error, missing group field, missing FASTA when required, etc.) are reported on stderr; other trees continue.
- Exit code 0: all trees succeeded.
- Exit code 1: bad arguments / empty input directory.
- Exit code 2: at least one tree failed.

18 Worked numerical sketch

Suppose dsr $= 6.5 \times 10^{-5}$ and filename DPI $= 14$. Then

$$\tau = 14 \times 6.5 \times 10^{-5} = 9.1 \times 10^{-4}, \qquad \text{etd} = 1.82 \times 10^{-3}.$$

A tip set with mean pairwise path distance $\overline{\delta} = 2.1 \times 10^{-3}$ has

$$\widehat{\text{DPI}} = \text{round}\left(\frac{2.1 \times 10^{-3}}{2 \times 6.5 \times 10^{-5}}\right) = \text{round}(16.15) = 16 > 14,$$

so Auto will attempt to split it (subject to size and bipartition constraints). After a successful split into children with DPI ≤ 14, Auto stops on those leaves.

19 File map

Path	Role
`tree_clustering.py`	Implementation (this guide)
`latex/tree_clustering_guide.tex`	This document
`latex/images/`	Example SVG/PDF figures
`latex/Makefile`	Build PDF via `pdflatex`

20 Versioning note

If script and document disagree, treat `tree_clustering.py` as authoritative and update this guide accordingly.

A Colour palette (clusters)

Cluster id $c \geq 0$ uses palette entry $c \bmod 14$. Hex codes below are printed in the corresponding colour, with a swatch:

Index	Hex
0	# ff00ff
1	# 3b82f6
2	# 10b981
3	# f59e0b
4	# 8b5cf6
5	# 06b6d4
6	# 84cc16
7	# f97316
8	# 6366f1
9	# 14b8a6
10	# 64748b
11	# fbbf24
12	# a855f7
13	# 22c55e

Group colours start at palette index 1 (group $g \rightarrow$ index $(g + 1) \bmod 14$). Noise uses # 9ca3af.

References

[1] Brandon F. Keele, Elena E. Giorgi, Jesus F. Salazar-Gonzalez, Julie M. Decker, Kimmy T. Pham, Maria G. Salazar, Chuanxi Sun, et al. Identification and characterization of transmitted and early founder virus envelopes in primary HIV-1 infection. *Proceedings of the National Academy of Sciences of the United States of America*, 105(21):7552–7557, 2008. doi:10.1073/pnas.0802203105.